Range Check 예제 분석

개요

Range Check는 영지식증명에서 매우 중요한 기본 연산입니다. 특정 값이 주어진 범위 내에 있음을 증명하는 것은 많은 응용 프로그램에서 필수적입니다. 예를 들어, 나이가 18세 이상임을 증명하거나, 잔액이 충분함을 증명하는 등의 경우에 사용됩니다.

plonky2는 `BaseSumGate` 를 사용하여 효율적인 Range Check를 구현합니다. 이 문서에서는 `range_check.rs` 예제를 통해 Range Check의 동작 원리를 상세히 분석합니다.

Range Check 개념

Range Check는 다음을 증명합니다:

```
x < 2^n
```

여기서 `x` 는 검증하려는 값이고, `n` 은 비트 수입니다.

plonky2에서는 이를 **비트 분해(Bit Decomposition)** 방식으로 구현합니다:

1. 값 `x` 를 비트로 분해: `x = b_0 + b_1*2 + b_2*2^2 + ... + b_{n-1}*2^{n-1}`
2. 각 `b_i` 가 0 또는 1임을 검증
3. 분해된 비트들의 합이 원래 값 `x` 와 같음을 검증

예제 코드 전체 분석

```rust
use anyhow::Result;
```

```
use plonky2::field::types::Field;

use plonky2::iop::witness::{PartialWitness, WitnessWrite};

use plonky2::plonk::circuit_builder::CircuitBuilder;

use plonky2::plonk::circuit_data::CircuitConfig;

use plonky2::plonk::config::{GenericConfig, PoseidonGoldilocksConfig};



fn  main() -> Result<()> {

const D: usize = 2;

type C = PoseidonGoldilocksConfig;

type F = <C as GenericConfig<D>>::F;



let  config = CircuitConfig::standard_recursion_config();

let  mut  builder = CircuitBuilder::<F, D>::new(config);



// The secret value.

let  value = builder.add_virtual_target();



// Registered as a public input (even though it's secret) so we can print out th

builder.register_public_input(value);



let  log_max = 6;

builder.range_check(value, log_max);
```

```
let mut pw = PartialWitness::new();
pw.set_target(value, F::from_canonical_usize(42))?;

let data = builder.build::<C>();
let proof = data.prove(pw)?;

println!(
"Value {} is less than 2^{}",
proof.public_inputs[0], log_max,
);

data.verify(proof)
}
```

단계별 분석

1. 설정 및 초기화

```
const D: usize = 2;
type C = PoseidonGoldilocksConfig;
type F = <C as GenericConfig<D>>::F;

let config = CircuitConfig::standard_recursion_config();
```

```rust
let mut builder = CircuitBuilder::<F, D>::new(config);
```

- `D = 2` : Extension degree (Goldilocks Field의 2차 확장 사용)
- `PoseidonGoldilocksConfig` : Poseidon 해시와 Goldilocks Field 조합
- `standard_recursion_config()` : 재귀 증명에 적합한 기본 설정

2. 검증할 값 추가

```rust
let value = builder.add_virtual_target();
builder.register_public_input(value);
```

- `add_virtual_target()` : 회로에 변수 추가
- `register_public_input()` : 공개 입력으로 등록 (예제에서는 출력을 위해 사용)

3. Range Check 적용

```rust
let log_max = 6;
builder.range_check(value, log_max);
```

이 한 줄이 핵심입니다. `range_check(value, 6)` 은 `value < 2^6 = 64` 임을 증명합니다.

4. Witness 설정

```rust
let mut pw = PartialWitness::new();
pw.set_target(value, F::from_canonical_usize(42))?;
```

실제 값 42를 설정합니다. 42 < 64이므로 증명이 성공합니다.

5. 증명 생성 및 검증

```
let data = builder.build::<C>();
let proof = data.prove(pw)?;
data.verify(proof)
```

range_check 함수 내부 구현

`range_check` 함수는 매우 간단합니다:

```
pub fn range_check(&mut self, x: Target, n_log: usize) {
self.split_le(x, n_log);
}
```

실제 작업은 `split_le` 함수에서 수행됩니다.

split_le 함수 상세 분석

`split_le` 는 정수를 리틀 엔디안 비트 리스트로 분해합니다:

```
pub fn split_le(&mut self, integer: Target, num_bits: usize) -> Vec<BoolTarge
if num_bits == 0 {
return Vec::new();
}
let gate_type = BaseSumGate::<2>::new_from_config::<F>(&self.config);
let k = num_bits.div_ceil(gate_type.num_limbs);
let gates = (0..k)
```

```
.map(|_| self.add_gate(gate_type, vec![]))
.collect::<Vec<_>>();

let mut bits = Vec::with_capacity(num_bits);
for &gate in &gates {
for limb_column in gate_type.limbs() {
bits.push(BoolTarget::new_unsafe(Target::wire(gate, limb_column)));
}
}
for b in bits.drain(num_bits..) {
self.assert_zero(b.target);
}

let zero = self.zero();
let base = F::TWO.exp_u64(gate_type.num_limbs as u64);
let mut acc = zero;
for &gate in gates.iter().rev() {
let sum = Target::wire(gate, BaseSumGate::<2>::WIRE_SUM);
acc = self.mul_const_add(base, acc, sum);
}
self.connect(acc, integer);

self.add_simple_generator(WireSplitGenerator {
```

```
integer,
gates,
num_limbs: gate_type.num_limbs,
});


bits
}
```

split_le 동작 과정

1. BaseSumGate 생성

```
let gate_type = BaseSumGate::<2>::new_from_config::<F>(&self.config);
let k = num_bits.div_ceil(gate_type.num_limbs);
```

- `BaseSumGate::<2>` : Base-2 (이진) 분해를 위한 게이트
- `num_limbs` : 한 게이트가 처리할 수 있는 비트 수
- `k` : 필요한 게이트 개수

2. 비트 Target 수집

```
let mut bits = Vec::with_capacity(num_bits);
for &gate in &gates {
for limb_column in gate_type.limbs() {
bits.push(BoolTarget::new_unsafe(Target::wire(gate, limb_column)));
```

```
}
}
```

각 게이트의 limb wire를 비트로 수집합니다.

3. 초과 비트 제거

```
for b in bits.drain(num_bits..) {
self.assert_zero(b.target);
}
```

필요한 비트 수를 초과하는 비트는 0으로 설정합니다.

4. 재구성 검증

```
let base = F::TWO.exp_u64(gate_type.num_limbs as u64);
let mut acc = zero;
for &gate in gates.iter().rev() {
let sum = Target::wire(gate, BaseSumGate::<2>::WIRE_SUM);
acc = self.mul_const_add(base, acc, sum);
}
self.connect(acc, integer);
```

분해된 비트들을 다시 합쳐서 원래 값과 같은지 검증합니다:

```
acc = sum_0 + base * sum_1 + base^2 * sum_2 + ...
```

5. Generator 추가

```
self.add_simple_generator(WireSplitGenerator {
integer,
gates,
num_limbs: gate_type.num_limbs,
});
```

Witness 생성 시 비트 분해를 자동으로 수행하는 Generator를 추가합니다.

BaseSumGate 동작 원리

`BaseSumGate<B>` 는 Base-B 분해를 위한 게이트입니다. Range Check에서는 `B=2` 를 사용합니다.

제약조건

BaseSumGate는 다음 제약조건들을 검증합니다:

1. 합 검증

```
computed_sum = reduce_with_powers(&limbs, B)
constraint: computed_sum - sum = 0
```

limb들의 가중합이 sum과 같은지 검증:

```
sum = limb_0 + limb_1*B + limb_2*B^2 + ... + limb_{n-1}*B^{n-1}
```

2. Range 검증

```
for limb in limbs {
constraint: (limb - 0) * (limb - 1) * ... * (limb - (B-1)) = 0
}
```

각 limb가 `{0, 1, ..., B-1}` 범위 내에 있는지 검증합니다.

`B=2` 인 경우:

```
(limb - 0) * (limb - 1) = 0
```

이는 `limb ∈ {0, 1}` 을 의미합니다.

제약조건의 수학적 의미

Range 검증 제약조건은 **Lagrange 보간**의 원리를 사용합니다:

- 다항식 `P(x) = (x-0)(x-1)...(x-(B-1))` 는 `x ∈ {0,1,...,B-1}` 일 때만 0입니다
- 따라서 `P(limb) = 0` 이면 `limb` 는 반드시 허용된 범위 내에 있습니다

WireSplitGenerator 분석

Generator는 Witness 생성 시 자동으로 비트 분해를 수행합니다:

```
fn run_once(
&self,
witness: &PartitionWitness<F>,
out_buffer: &mut GeneratedValues<F>,
) -> Result<()> {
```

```
let mut integer_value = witness.get_target(self.integer).to_canonical_u64();

for &gate in &self.gates {
let sum = Target::wire(gate, BaseSumGate::<2>::WIRE_SUM);

let mut truncated_value = integer_value;
if self.num_limbs < 64 {
truncated_value = integer_value & ((1 << self.num_limbs) - 1);
integer_value >>= self.num_limbs;
} else {
integer_value = 0;
};

out_buffer.set_target(sum, F::from_canonical_u64(truncated_value))?;
}

Ok(())
}
```

동작 과정

1. 원래 정수 값을 가져옵니다
2. 각 게이트에 대해:

- 하위 `num_limbs` 비트를 추출: `truncated_value = integer_value & ((1 << num_limbs) - 1)`
- 해당 비트들을 게이트의 sum wire에 설정
- 남은 비트들로 이동: `integer_value >>= num_limbs`

예를 들어, `integer_value = 42 = 0b101010`, `num_limbs = 3` 인 경우:

- 첫 번째 게이트: `0b010 = 2`
- 두 번째 게이트: `0b101 = 5`

실행 결과

예제를 실행하면 다음과 같은 출력을 얻습니다:

```
Value 42 is less than 2^6
```

증명 크기 및 성능

Range Check는 매우 효율적입니다:

- **Gate 수**: `⌈num_bits / num_limbs⌉` 개의 BaseSumGate
- **제약조건 수**: 각 게이트당 `1 + num_limbs` 개
- 1개: 합 검증
- `num_limbs` 개: 각 비트의 범위 검증
- **Degree**: `B` (Base-2의 경우 2)

예를 들어, 6비트 Range Check의 경우:

- Goldilocks Field에서 `num_limbs ≈ 32` (한 게이트가 최대 32비트 처리)
- 필요한 게이트: 1개

- 제약조건: 1 + 6 = 7개

다른 예제와의 비교

Fibonacci 예제와의 차이점

| 특징 | Fibonacci | Range Check |

|------|----------|----------|

| 주요 연산 | 덧셈 | 비트 분해 |

| 사용 Gate | ArithmeticGate | BaseSumGate |

| 제약조건 | 선형 (degree 1) | 다항식 (degree B) |

| 목적 | 계산 증명 | 범위 증명 |

응용 사례

Range Check는 다음과 같은 경우에 사용됩니다:

1. **나이 검증**: 나이가 18세 이상임을 증명
2. **잔액 검증**: 잔액이 충분함을 증명 (오버플로우 방지)
3. **비트 연산**: 비트 단위 연산의 정확성 증명
4. **Lookup Table**: 인덱스가 테이블 범위 내에 있음을 증명

split_low_high 함수

Range Check와 관련된 또 다른 유용한 함수는 `split_low_high` 입니다:

```
pub fn split_low_high(&mut self, x: Target, n_log: usize, num_bits: usize) ->
let low = self.add_virtual_target();
```

```
let  high = self.add_virtual_target();

self.add_simple_generator(LowHighGenerator {
integer: x,
n_log,
low,
high,
});

self.range_check(low, n_log);
self.range_check(high, num_bits - n_log);

let  pow2 = self.constant(F::from_canonical_u64(1 << n_log));
let  comp_x = self.mul_add(high, pow2, low);
self.connect(x, comp_x);

(low, high)
}
```

이 함수는 정수를 상위 비트와 하위 비트로 분리합니다:

```
x = low + high * 2^n_log
```

여기서 `low < 2^n_log` 입니다.

사용 예시

64비트 정수를 32비트 두 개로 분리:

```
let (low_32, high_32) = builder.split_low_high(value, 32, 64);
```

최적화 기법

1. Gate 재사용

plonky2는 동일한 설정의 BaseSumGate를 재사용하여 효율성을 높입니다.

2. 병렬 처리

여러 Range Check를 동시에 수행할 때 병렬 처리가 가능합니다.

3. Lookup Table 대안

매우 작은 범위(예: 0-255)의 경우, Lookup Table을 사용하는 것이 더 효율적일 수 있습니다.

요약

Range Check는 영지식증명에서 필수적인 기본 연산입니다. plonky2는 다음과 같은 방식으로 효율적으로 구현합니다:

1. **비트 분해**: 값을 비트로 분해
2. **BaseSumGate**: 각 비트가 0 또는 1임을 검증
3. **재구성 검증**: 분해된 비트들의 합이 원래 값과 같음을 검증
4. **Generator**: Witness 생성 자동화

이 방식은 간단하면서도 효율적이며, 다양한 응용 프로그램에서 활용될 수 있습니다.

관련 문서

- 02-circuit-building.md: CircuitBuilder 기본 사용법
- 07-example-fibonacci.md: Fibonacci 예제 분석
- 09-example-square-root.md: Custom Generator 작성법

문서 네비게이션

- **이전**: 07. Fibonacci 예제 - 기본 회로 구성
- **다음**: 09. Square Root 예제 - Custom Generator 사용
- **관련 문서**:
- 02. 회로 구성 프로세스 - CircuitBuilder 기본
- 07. Fibonacci 예제 - 기본 예제
- 09. Square Root 예제 - 고급 예제